PYTHON PROGRAMMING PART

Student Record and Academic Management System

Student Name	Nikita Dnyaneshwar Udavant
PRN	2127UDSF1031
Subject	Python Programming

Data Organization using Python Collections

1. String Program — Student Details

Strings store text such as a student's name and department.

```
name = input("Enter student name: ")
department = input("Enter department: ")
print("Student Name:", name)
print("Department:", department)
print("Name in uppercase:", name.upper())
```

2. List Program — Average Marks

```
marks = [70, 80, 90]
total = 0
for mark in marks:
    total = total + mark
average = total / len(marks)
print("Marks:", marks)
print("Average:", average)
```

3. Tuple Program — Fixed Information

```
student_info = (101, "Asha Patil", "IT")
print("Roll Number:", student_info[0])
print("Name:", student_info[1])
print("Department:", student_info[2])
```

4. Set Program — Unique Departments

```
departments = {"IT", "Computer", "IT", "ENTC"}
print("Departments:", departments)
departments.add("Mechanical")
print("After adding:", departments)
```

Note: Set display order may vary.

5. Dictionary Program — Student Profile

```
student = {
    "roll": 101,
    "name": "Asha Patil",
    "department": "IT",
    "marks": [70, 80, 90],
    "attendance": 85
}
print("Name:", student["name"])
print("Department:", student["department"])
print("Marks:", student["marks"])
print("Attendance:", student["attendance"])
```

6. Integrated Student Record Management System

This menu-driven program uses a list for records, dictionaries for student details, lists for subjects and marks, a tuple for fixed registration information, and sets for unique departments and subjects.

```
students = []
departments = set()
all_subjects = set()

def find_student(roll):
    for student in students:
        if student["roll"] == roll:
            return student
    return None

def add_student():
    roll = input("Enter roll number: ")
    if find_student(roll) is not None:
        print("Roll number already exists.")
        return
    name = input("Enter name: ")
    department = input("Enter department: ")
    email = input("Enter email: ")
    address = input("Enter address: ")
    subjects = input("Enter subjects separated by commas: ").split(",")
    marks_text = input("Enter marks separated by commas: ").split(",")
    marks = []
    for value in marks_text:
        marks.append(float(value.strip()))
    fixed_info = (roll, department)
    student = {
        "roll": roll,
        "name": name,
        "department": department,
        "email": email,
        "address": address,
        "subjects": subjects,
        "marks": marks,
        "attendance": input("Enter attendance percentage: "),
        "registration_info": fixed_info
    }
    students.append(student)
    departments.add(department)
    for subject in subjects:
        all_subjects.add(subject.strip())
    print("Student added successfully.")

def search_student():
    roll = input("Enter roll number to search: ")
    student = find_student(roll)
    if student is None:
        print("Student not found.")
    else:
        print(student)

def update_student():
    roll = input("Enter roll number to update: ")
    student = find_student(roll)
    if student is None:
        print("Student not found.")
    else:
        student["name"] = input("Enter updated name: ")
        student["department"] = input("Enter updated department: ")
        departments.add(student["department"])
        print("Student updated.")

def delete_student():
    roll = input("Enter roll number to delete: ")
    student = find_student(roll)
    if student is None:
        print("Student not found.")
    else:
        students.remove(student)
        print("Student deleted.")

def display_students():
    if len(students) == 0:
        print("No student records.")
    else:
        for student in students:
            print(student)

def average_marks():
    roll = input("Enter roll number: ")
    student = find_student(roll)
```

```
    if student is None:
        print("Student not found.")
    elif len(student["marks"]) == 0:
        print("No marks available.")
    else:
        total = 0
        for mark in student["marks"]:
            total = total + mark
        print("Average marks:", total / len(student["marks"]))

def highest_scorer():
    if len(students) == 0:
        print("No student records.")
        return
    highest = None
    highest_average = -1
    for student in students:
        if len(student["marks"]) > 0:
            total = 0
            for mark in student["marks"]:
                total = total + mark
            average = total / len(student["marks"])
            if average > highest_average:
                highest_average = average
                highest = student
    if highest is not None:
        print("Highest scorer:", highest["name"], highest_average)

def list_by_department():
    dept = input("Enter department: ").lower()
    found = False
    for student in students:
        if student["department"].lower() == dept:
            print(student["name"], student["roll"], student["department"])
            found = True
    if not found:
        print("No student found in this department.")

while True:
    print("\n1.Add  2.Search  3.Update  4.Delete")
    print("5.Display  6.Average  7.Highest Scorer")
    print("8.List by Department  9.Show Unique Values  10.Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        add_student()
    elif choice == "2":
        search_student()
    elif choice == "3":
        update_student()
    elif choice == "4":
        delete_student()
    elif choice == "5":
        display_students()
    elif choice == "6":
        average_marks()
    elif choice == "7":
        highest_scorer()
    elif choice == "8":
        list_by_department()
    elif choice == "9":
        print("Departments:", departments)
        print("Subjects:", all_subjects)
    elif choice == "10":
        break
    else:
        print("Invalid choice. Try again.")
```

7. How to Run

1. Open Python IDLE or another Python editor. 2. Create a file named student_record.py. 3. Copy the integrated code into it. 4. Run the file. 5. Choose an option from the menu and enter the requested details.

8. Testing Checklist

- Add a new student and display the record.
- Try a duplicate roll number.
- Search for an existing and a missing record.
- Update and delete a record.
- Calculate average marks and find the highest scorer.
- List students by department.

• Display unique departments and subjects.

Note: Enter marks as numbers separated by commas, for example 70,80,90. This simple program expects valid numeric input.